APRENDIZAGEM PROFUNDA

Luís Morgado

ISEL-DEI

APRENDIZAGEM POR REFORÇO PROFUNDA

Problema da dimensionalidade dos espaços de estados

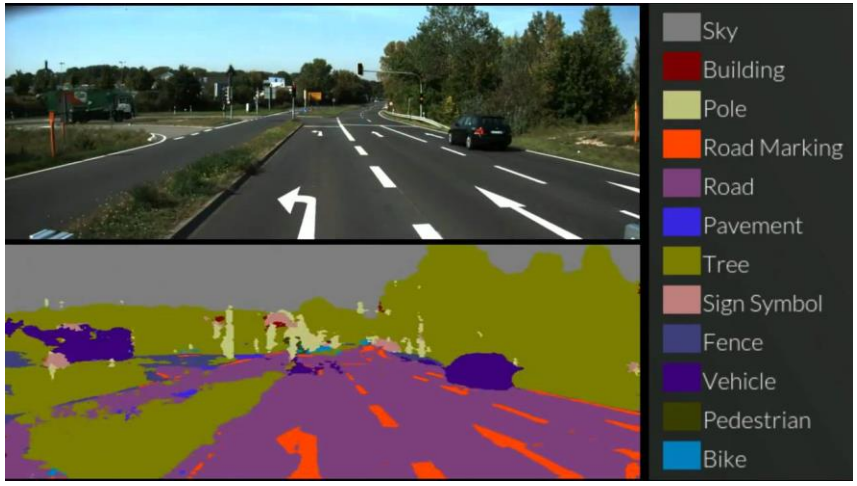
- Os espaços de estados de problemas concretos têm tipicamente uma dimensão muito elevada
  - A resolução desses problemas por algoritmos de aprendizagem por reforço não é viável em tempo útil
  - Exemplo:
    - Treino de sistemas autónomos para realização de tarefas complexas, como manipulação de objectos ou navegação em ambientes desconhecidos

APRENDIZAGEM POR REFORÇO PROFUNDA

Problema: Complexidade dos espaços de estados

- Para problemas com **espaços de estados de elevada complexidade** (*problema da dimensionalidade*) a aprendizagem torna-se inviável se todos os pares estado-acção forem processados de forma exaustiva
  - Necessidade de **abstracção e generalização**
- A alternativa é utilizar métodos de **aproximação de funções**, em particular **redes neuronais artificiais profundas**, para aproximar a função Q
  - *Aprendizagem por reforço profunda*

Exemplo: Condução automática em veículos autónomos



[NVidia, 2015]

APRENDIZAGEM POR REFORÇO PROFUNDA

Meio de lidar com o problema da dimensionalidade dos espaços de estados e de acções

- **Integração de tecnologias**
  - **Redes neuronais artificiais**
    - Para abordar o problema da representação de espaços de estados complexos
    - Capacidade de generalização / abstracção
  - **Aprendizagem por reforço**
    - Para abordar o problema da estimação de valor e tomada de decisão

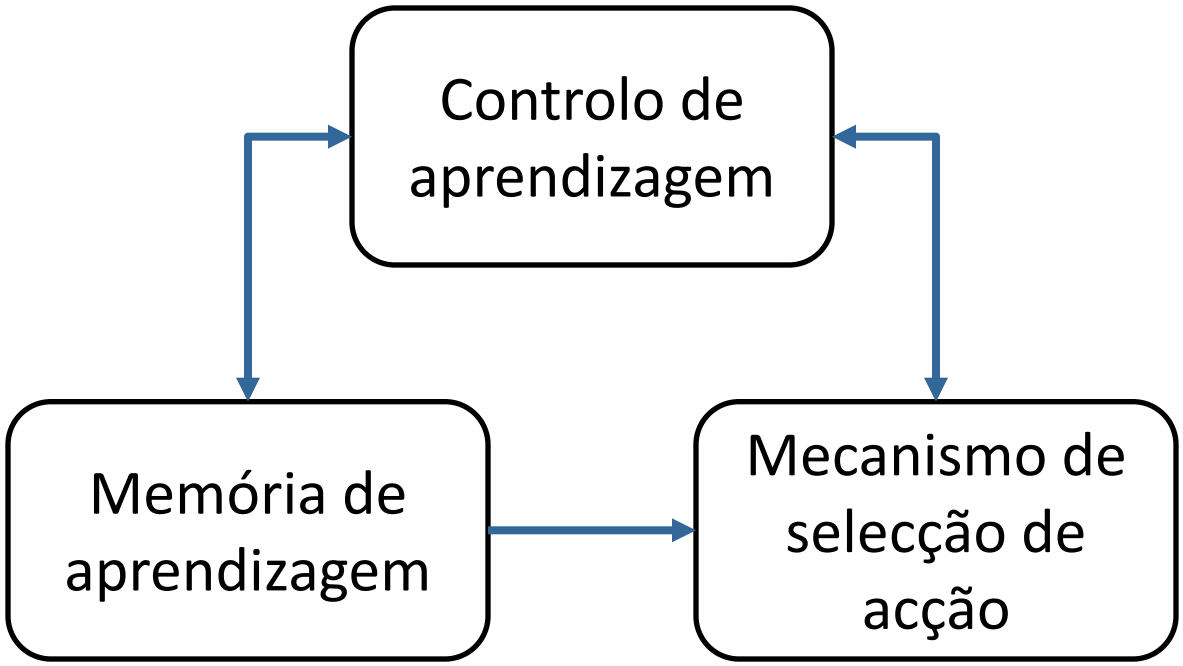
APRENDIZAGEM POR REFORÇO PROFUNDA

Integração de aprendizagem por reforço com redes neuronais artificiais

- **Redes neuronais artificiais**
  - Memória de aprendizagem
    - Realiza a função $Q(s,a)$
    - Generalização/abstracção de estado
- **Aprendizagem por reforço**
  - Controlo do processo de aprendizagem
    - Estratégias de selecção de acção
    - Métodos de actualização de estimativas de valor
    - Políticas de tomada de decisão

APRENDIZAGEM POR REFORÇO

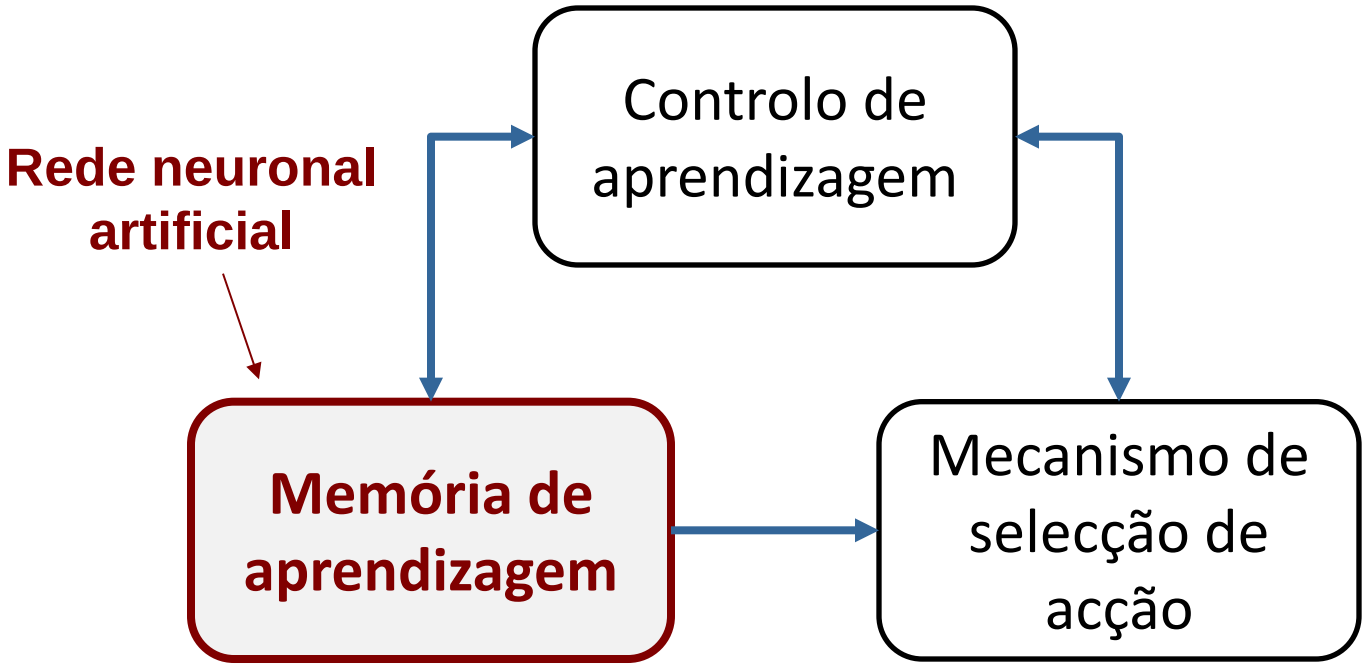
Mecanismo geral de aprendizagem



Fluxo de informação

APRENDIZAGEM POR REFORÇO PROFUNDA

Mecanismo geral de aprendizagem



Redução de complexidade por aproximação da função de estimativa de valor $Q(s, a)$

ALGORITMO *Q-LEARNING*

Problema: Em cada passo de aprendizagem apenas está disponível um exemplo de treino, no entanto, uma rede neuronal artificial requer um conjunto de exemplos de treino

1. Iniciar $Q(s,a)$
2. Repetir (por cada episódio)
3. Iniciar s
4. Repetir (por cada passo)
5. Escolher a de acordo com s com base numa política derivada de Q (por exemplo, ε-*greedy*)
6. Executar a acção a, observar r e s'
7. Actualizar Q:
$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$
8. Actualizar $s \leftarrow s'$
9. Até s ser um estado terminal

APRENDIZAGEM POR REFORÇO PROFUNDA

Memória de aprendizagem baseada em redes neuronais artificiais

- **Problema:**
  - **Aprendizagem em redes neuronais artificiais**
    - Requer um conjunto de exemplos de treino
    - Requer a iteração repetida sobre os dados de treino para convergência da aprendizagem
    - Requer uma função de erro (*perda*)
  - **Aprendizagem por reforço**
    - Os exemplos surgem ao longo da aprendizagem
    - Os reforços obtidos ao longo do tempo determinam as estimativas de valor
- **Como integrar estes dois tipos de aprendizagem?**

APRENDIZAGEM COM MEMÓRIA DE EXPERIÊNCIA (*Experience Replay*)

Possibilita acelerar o processo de aprendizagem por iteração com base numa memória de experiência para além da interacção com o ambiente

- As experiências do agente são memorizadas numa **memória de experiência** ("***replay memory***")
- Num determinado instante t, a experiência do agente é definida por um tuplo:

$$e_t = (s_t, a_t, r_{t+1}, s_{t+1})$$

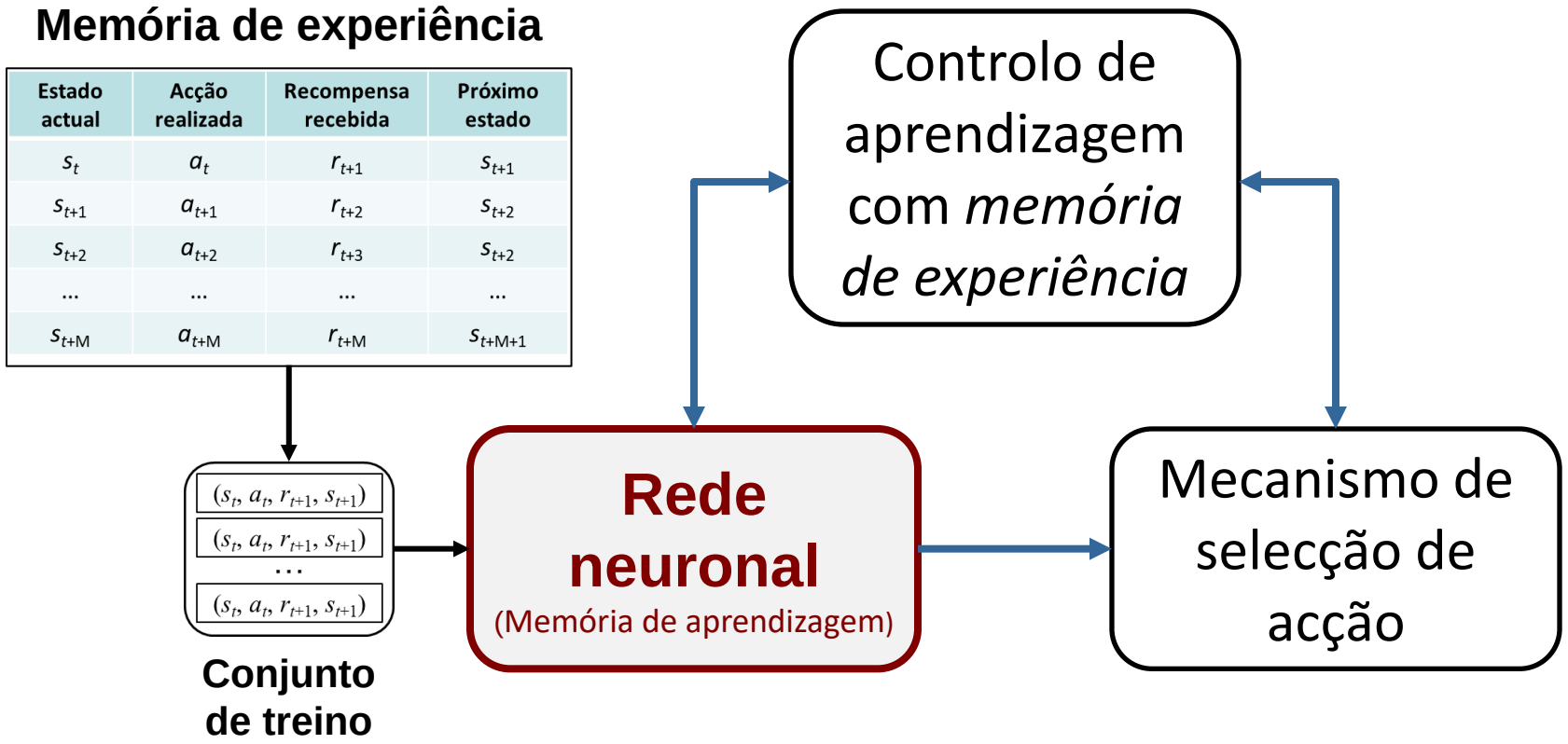
APRENDIZAGEM COM MEMÓRIA DE EXPERIÊNCIA

A memória de experiência é utilizada para actualizar as estimativas de valor através da recordação de experiências passadas

- Possibilita a actualização acelerada da estimativa de valor $Q(s,a)$ por iteração com base na memória de experiência para além da interacção com o ambiente
- Possibilita a integração com mecanismos de memória de aprendizagem baseados em redes neuronais artificiais
  - A memória de experiência fornece o conjunto de exemplos de treino

APRENDIZAGEM POR REFORÇO PROFUNDA

A memória de experiência fornece o conjunto de exemplos de treino necessário para a aprendizagem da rede neuronal



Q-LEARNING COM REDE NEURONAL

Formato geral da rede neuronal

Representação da função de estimativa de valor $Q(s, a)$

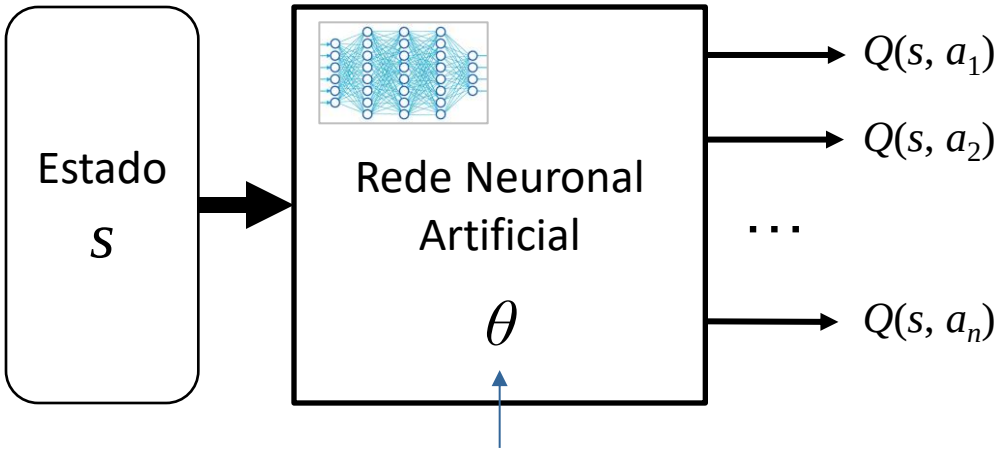
$$Q(s, a) \equiv \{\ Q(s, a_i)\ \mid a_i \in A(s)\ \}$$

Estimativa de valor $Q(s, a)$:

Rede neuronal

Entradas:

Representação de estado



Saídas:

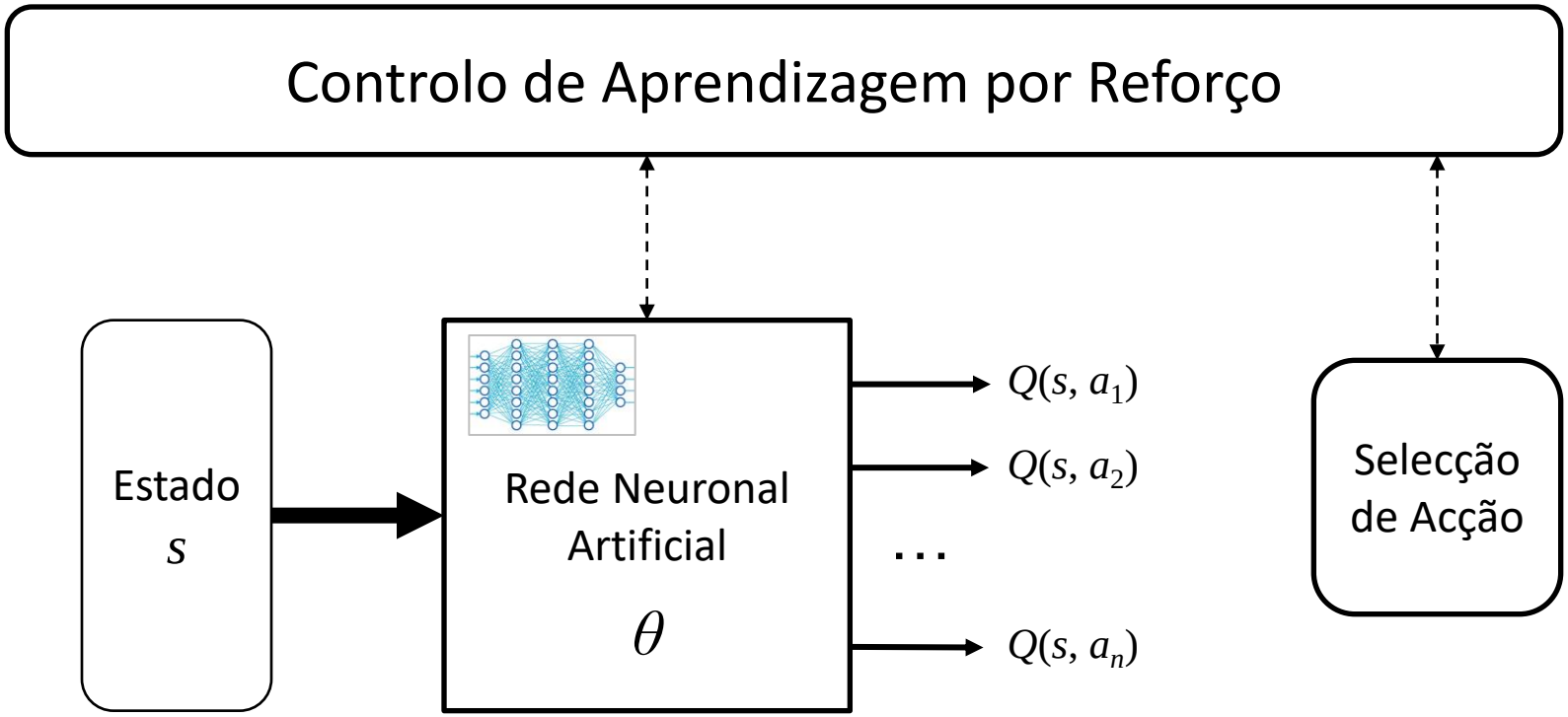
Estimativas de valor $Q(s, a_i)$ para cada acção definida

Parâmetros da rede:

$$\theta = \{\boldsymbol{W}^2, \boldsymbol{b}^2, \ldots, \boldsymbol{W}^{N-1}, \boldsymbol{b}^{N-1}, \boldsymbol{W}^N, \boldsymbol{b}^N\}$$

Q-LEARNING COM REDE NEURONAL

Mecanismo de aprendizagem



Memória de aprendizagem:

Rede neuronal para aproximação da função de estimativa de valor $Q(s,a)$

Suporte de abstracção e generalização

Q-LEARNING COM REDE NEURONAL E MEMÓRIA DE EXPERIÊNCIA

1. **Iniciar Q com parâmetros aleatórios θ**
2. Iniciar Memória de experiência $D \leftarrow \{\}$
3. Repetir (por cada episódio):
4. $s \leftarrow$ estado inicial
5. Repetir (por cada passo):
6. $a \leftarrow$ seleccionar acção (por exemplo, estratégia ε-*greedy*)
7. Executar a
8. Observar a recompensa r e o estado seguinte s'
9. Criar registo de experiência $e = (s, a, r, s')$
10. Actualizar memória de experiência D com e
11. *amostras* $\leftarrow$ Obter n Amostras da memória de experiência D
12. **Realizar uma iteração de optimização de $Q(s, a)$:**
13. **Definir a saída alvo $y = r + \gamma * \max_{a'} Q(s', a')$**
14. **para cada amostra $(s, a, r, s') \in$ *amostras***
15. Actualizar $s \leftarrow s'$

Q-LEARNING COM REDE NEURONAL

A aprendizagem por reforço pode ser instável ou mesmo divergente quando é utilizado um *aproximador de funções não linear*, como é o caso de uma **rede neural**, para representar a função de estimativa de valor $Q(s,a)$

Esta instabilidade pode ter várias causas, nomeadamente:

- Correlações presentes na sequência de observações
- O facto de pequenas actualizações da função $Q(s,a)$ poderem alterar significativamente a política
- Existência de correlações entre os valores de estado-acção $Q(s,a)$ e os valores alvo de retorno $r + \gamma \max_{a'} Q(s',a')$

APRENDIZAGEM POR REFORÇO PROFUNDA

Mecanismo *Q-Learning Profundo* (*Deep Q-Learning*)

- Utilização de memória de experiência (*Experience Replay Memory*) para treino da rede
- **Problema**:
  - **Função alvo $Q(s,a)$ depende da própria rede**
    - Possível instabilidade do processo de aprendizagem
- **Solução**:
  - **Utilização de duas instâncias de rede distintas**
    - **Instância de previsão** (actualização por cada passo de aprendizagem por reforço)
    - **Instância alvo** (actualização periódica)
      - Mantém o valor alvo fixo por várias passos de modo a estabilizar a aprendizagem de valor
    - Actualizadas em momentos diferentes, para estabilidade do processo de aprendizagem

APRENDIZAGEM POR REFORÇO PROFUNDA

Mecanismo *Q-Learning Profundo* (*Deep Q-Learning*)

Registo de experiência

$$e_t = (s_t, a_t, r_t, s_{t+1})$$

Memória de experiência

$$D_t = \{e_1, \ldots, e_t\}$$

Amostragem da memória de experiência $U(D)$

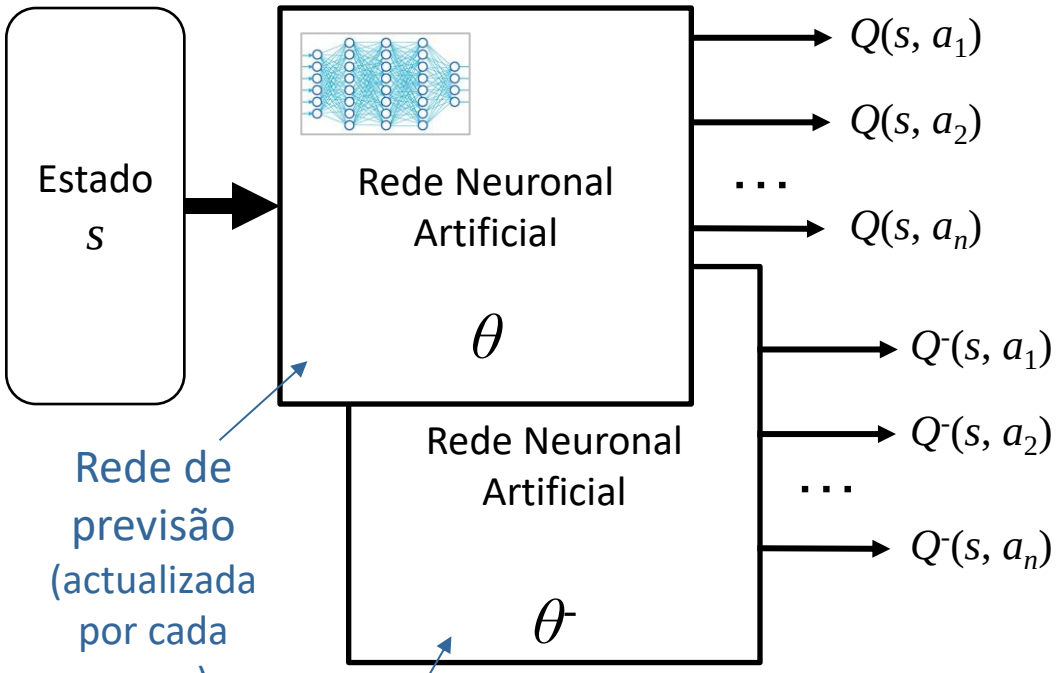
$$(s, a, r, s') \sim U(D)$$

Amostra de experiência memorizada

Operação de amostragem

Memória de aprendizagem:

Duas redes Q (previsão) e Q^- (alvo) com parâmetros θ e θ^-



Rede de previsão (actualizada por cada passo)

Rede alvo (actualizada por cada *C* passos)

Mantém o valor alvo fixo por várias passos de modo a estabilizar a aprendizagem de valor

ALGORITMO DE RETROPROPAGAÇÃO

Actualização de pesos e pendores com base na **retropropagação do erro de saída**

α – Taxa de aprendizagem

$$\Delta w_{ij}^{n} = -\alpha \frac{\partial L}{\partial w_{ij}^{n}}$$

$$\Delta b_{j}^{n} = -\alpha \frac{\partial L}{\partial b_{j}^{n}}$$

$$w_{ij}^{n} \leftarrow w_{ij}^{n} + \Delta w_{ij}^{n}$$

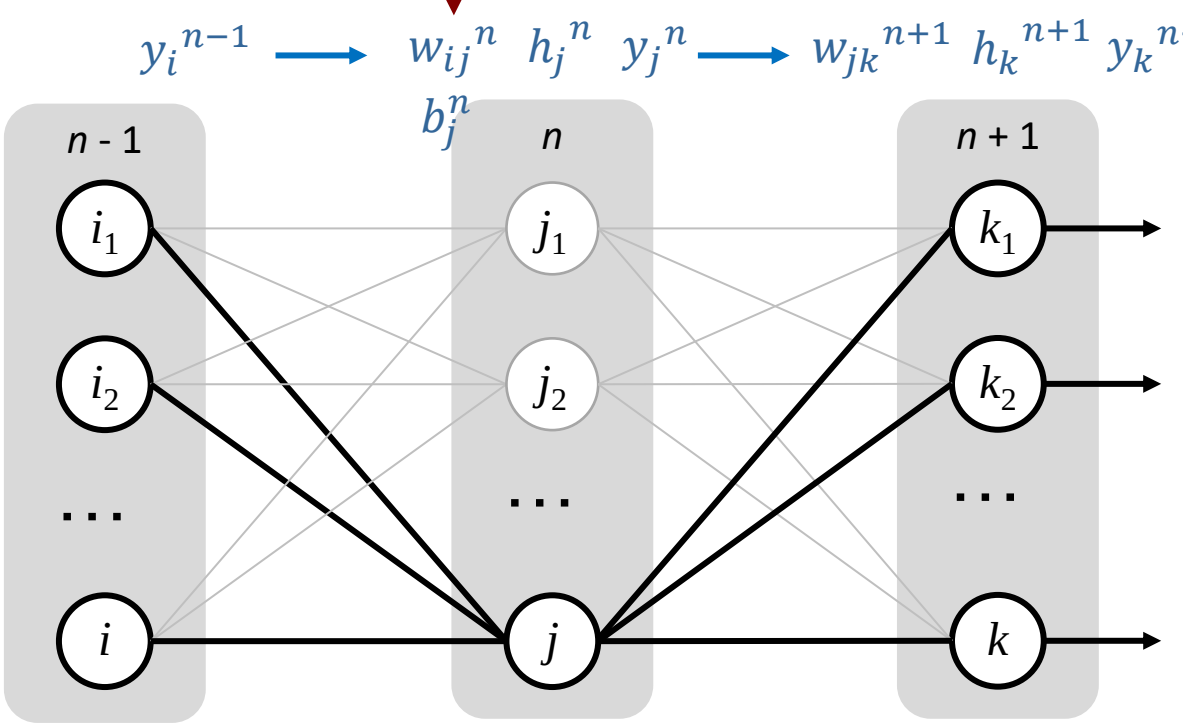
$$b_{j}^{n} \leftarrow b_{j}^{n} + \Delta b_{j}^{n}$$

Retropropagação do erro de saída

L
Perda (*Loss*) de saída

y
Saídas alvo

O que se pretende aprender



ALGORITMO *Q-LEARNING*

Estimativa de retorno *R* (valor estimado de realizar a acção *a* num estado *s*) **considerando a acção *a'* correspondente à melhor estimativa *Q(s',a')***

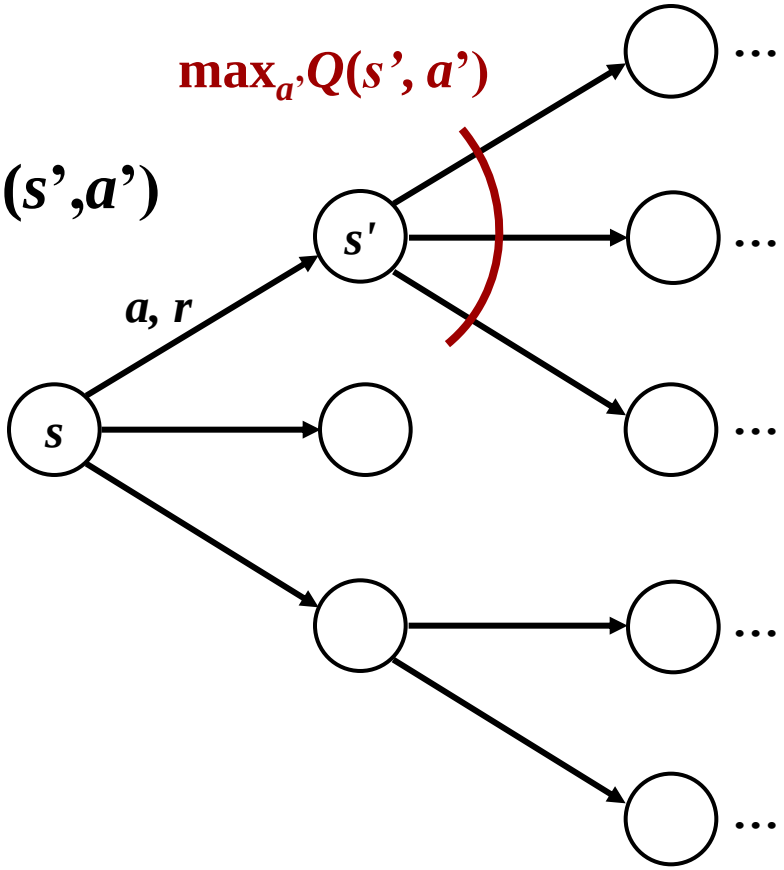
$$R = r + \gamma \max_a Q(s',a')$$

$\downarrow$

y

Saídas alvo

Objectivo: Aprender o retorno a longo prazo



Das várias acções possíveis no estado s' é escolhida para propagação de valor a que maximiza a respectiva estimativa de valor $\max_a Q(s',a')$

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_a Q(s',a') - Q(s,a)]$$

APRENDIZAGEM POR REFORÇO PROFUNDA

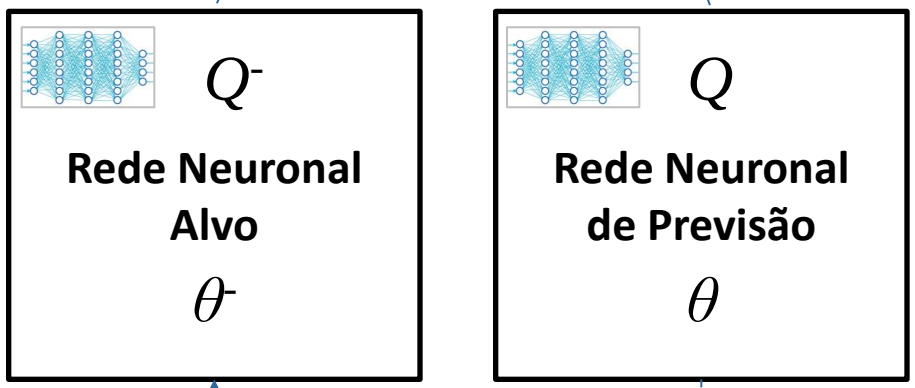
Iteração de optimização de *Q*(*s*, *a*)

Definir a saída alvo $y = r + \gamma \max_{a'} Q(s', a')$ para cada amostra da amostragem U da memória de experiência D: $(s, a, r, s') \in U(D)$

$$L_i(s, a, r, s') = \left(\underbrace{r + \gamma \max_{a'} Q^{-}(s', a')}_{\text{Valor alvo}} - \underbrace{Q(s, a)}_{\text{Previsão}} \right)^2$$

Erro (perda) para cada amostra na iteração i

Apenas o erro da saída *Q*(*s*,*a*) é calculado, as saídas correspondentes às restantes acções não são consideradas ($L_i = 0$)

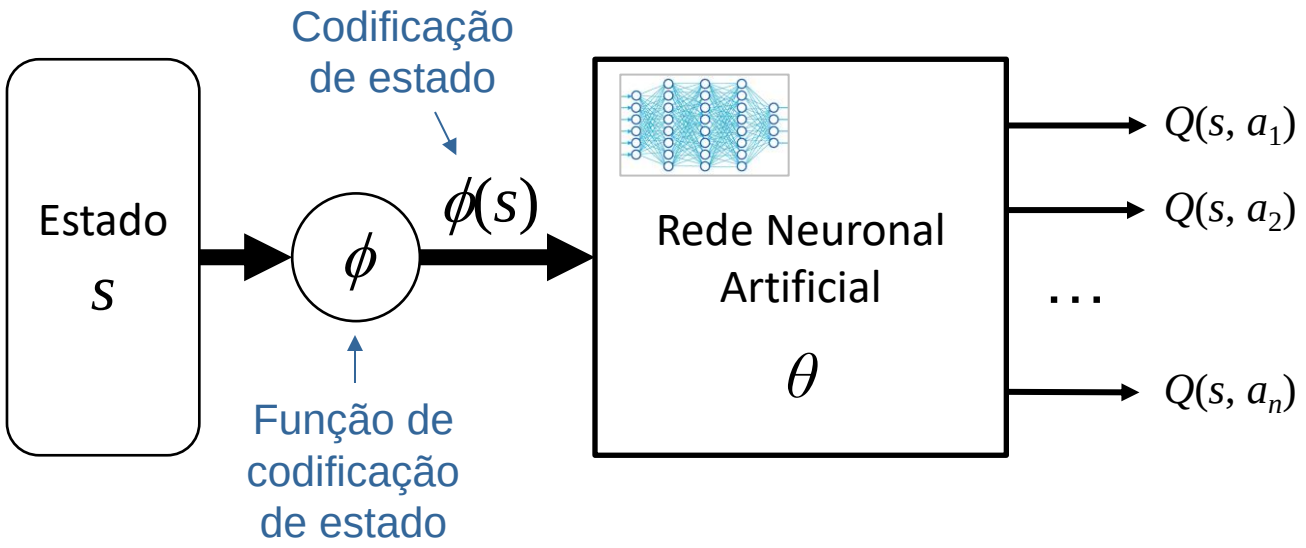


Actualização periódica dos parâmetros por cada C passos

MECANISMO *Q-LEARNING PROFUNDO*

Codificação de entrada da rede neuronal

No caso geral, para utilização de uma rede neuronal como memória de aprendizagem, a representação de estado s deve ser codificada através de uma função de codificação ϕ adequada à representação de entrada da arquitectura de rede neuronal utilizada



ALGORITMO *Q-LEARNING PROFUNDO*

1. **Iniciar a rede de previsão Q com parâmetros aleatórios θ**
2. **Iniciar a rede alvo Q^- com parâmetros aleatórios θ^-**
3. Iniciar Memória de experiência $D \leftarrow \{\}$
4. Repetir (por cada episódio):
5. $s \leftarrow$ estado inicial
6. Repetir (por cada passo):
7. $a \leftarrow$ seleccionar acção (por exemplo, estratégia ε-*greedy*)
8. Executar a
9. Observar a recompensa r e o estado seguinte s'
10. Criar registo de experiência $e = (s, a, r, s')$
11. Actualizar memória de experiência D com e
12. *amostras* $\leftarrow$ Obter n Amostras da memória de experiência D
13. **Realizar uma iteração de optimização de $Q(\phi(s), a, \theta)$:**
14. **Definir a saída alvo $y = r + \gamma * \max_{a'} Q^-(\phi(s'), a', \theta^-)$**
15. **para cada amostra $(s, a, r, s') \in$ *amostras***
16. **Actualizar a cada C passos $Q^- \leftarrow Q$**
17. Actualizar $s \leftarrow s'$

APRENDIZAGEM POR REFORÇO PROFUNDA

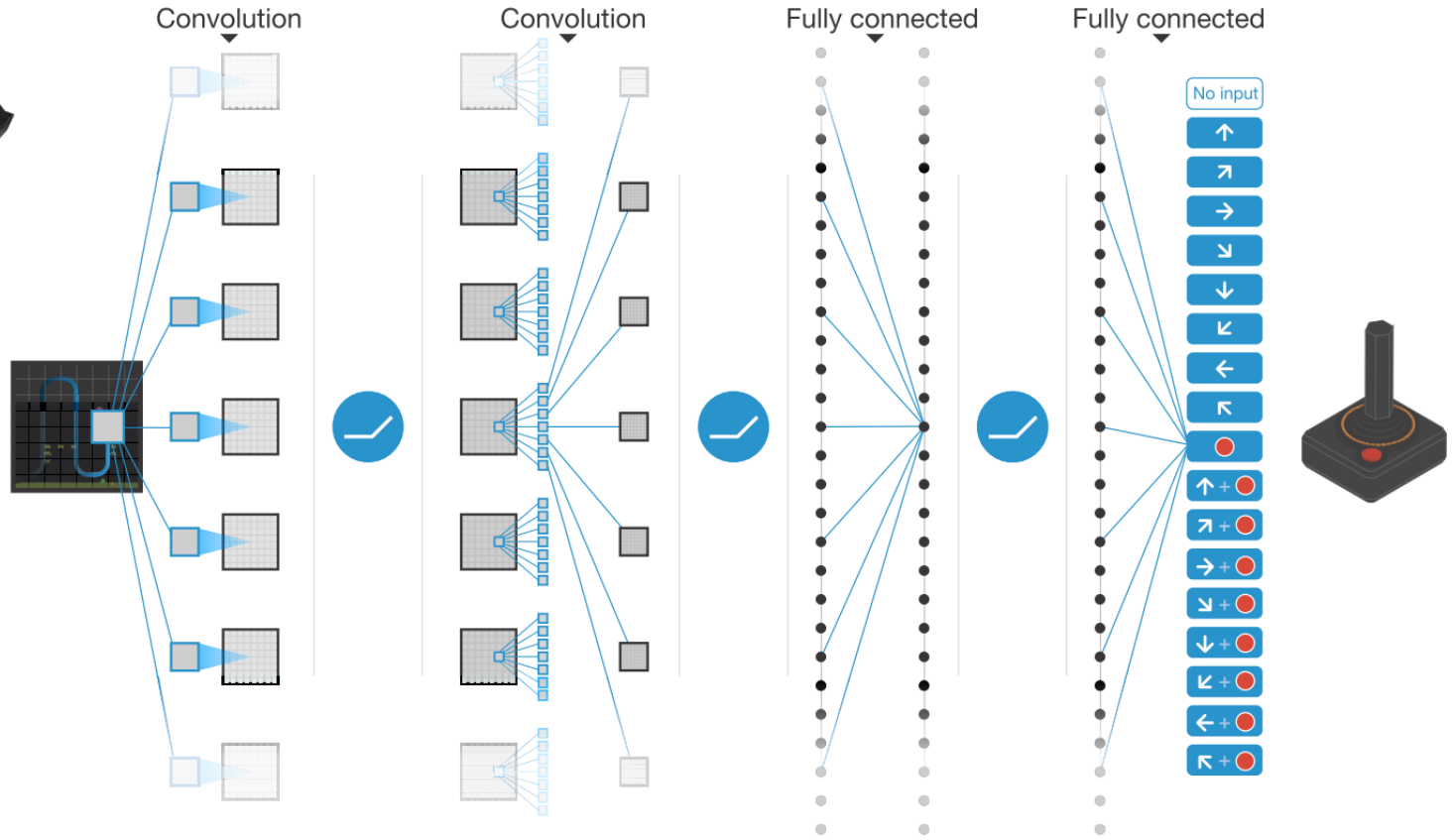
Exemplo de arquitectura da rede neuronal

Human-level control through deep reinforcement learning [Mnih *et al*., 2015]

Domínio de teste: Jogos Atari-2600 clássicos, agente com acesso apenas à imagem do ecrã e à pontuação para efeitos de representação de estado e geração de reforço



Consola Atari-2600



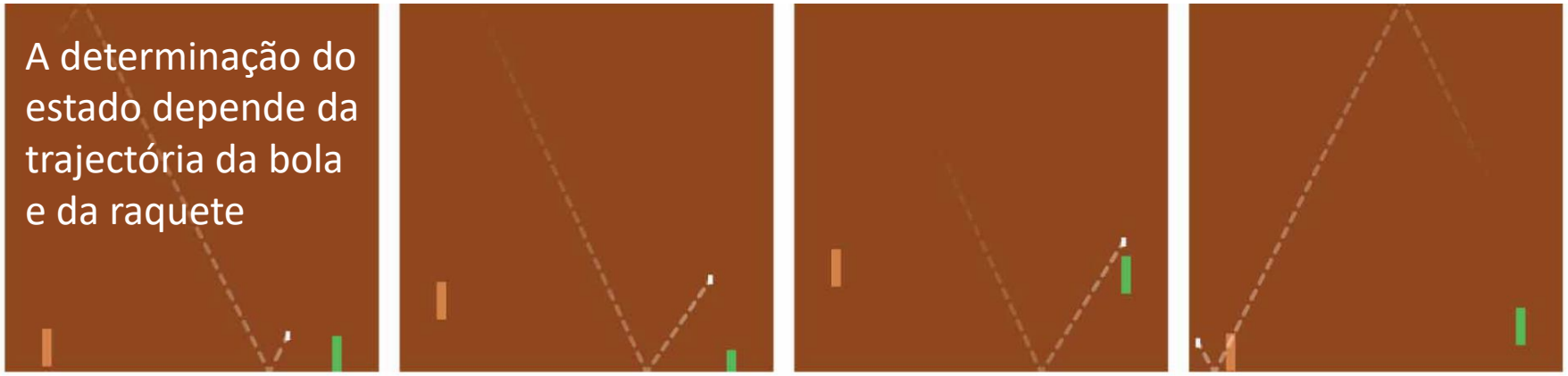
REPRESENTAÇÃO DE ESTADO

Problema da ambiguidade de estado (*aliasing*)

- Ocorre quando diferentes estados do ambiente são indistinguíveis para o agente
  - Vários estados diferentes podem ser percebidos como o mesmo estado devido à representação utilizada

Exemplo:

No jogo *Pong* se o estado for representado com base nas posições da bola e da raquete, várias situações do jogo originam a mesma representação



[Mnih *et al*., 2015]

REPRESENTAÇÃO DE ESTADO

Problema da ambiguidade de estado (*aliasing*)

Solução:

- Incluir na representação de estado a informação suficiente para garantir a adequada discriminação de estado

Exemplo:

No jogo *Pong* a representação de estado deve incluir a situação anterior do ambiente e acção realizada, de modo a expressar a trajectória da bola e da raquete:

$s_{t+1} = s_t, a_t, x_{t+1}$

x_{t+1} – Imagem do ambiente após a realização da acção a_t

APRENDIZAGEM POR REFORÇO PROFUNDA

Algoritmo *Deep Q-Learning* [Mnih *et al*., 2015]

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^{-} = \theta$

For episode $= 1, M$ **do**

  Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

  For $t = 1$,T **do**

    With probability ε select a random action a_t

    otherwise select $a_t = \text{argmax}_a Q(\phi(s_t), a; \theta)$

    Execute action a_t in emulator and observe reward r_t and image x_{t+1}

    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

    Set $y_j = \begin{cases} r_j & \text{if episode terminates at step j+1} \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^{-}) & \text{otherwise} \end{cases}$

    Perform a gradient descent step on $\left(y_j - Q(\phi_j, a_j; \theta)\right)^2$ with respect to the network parameters θ

    Every C steps reset $\hat{Q} = Q$

  End For

End For

$\phi(s_t)$

Função de pré-processamento de estado

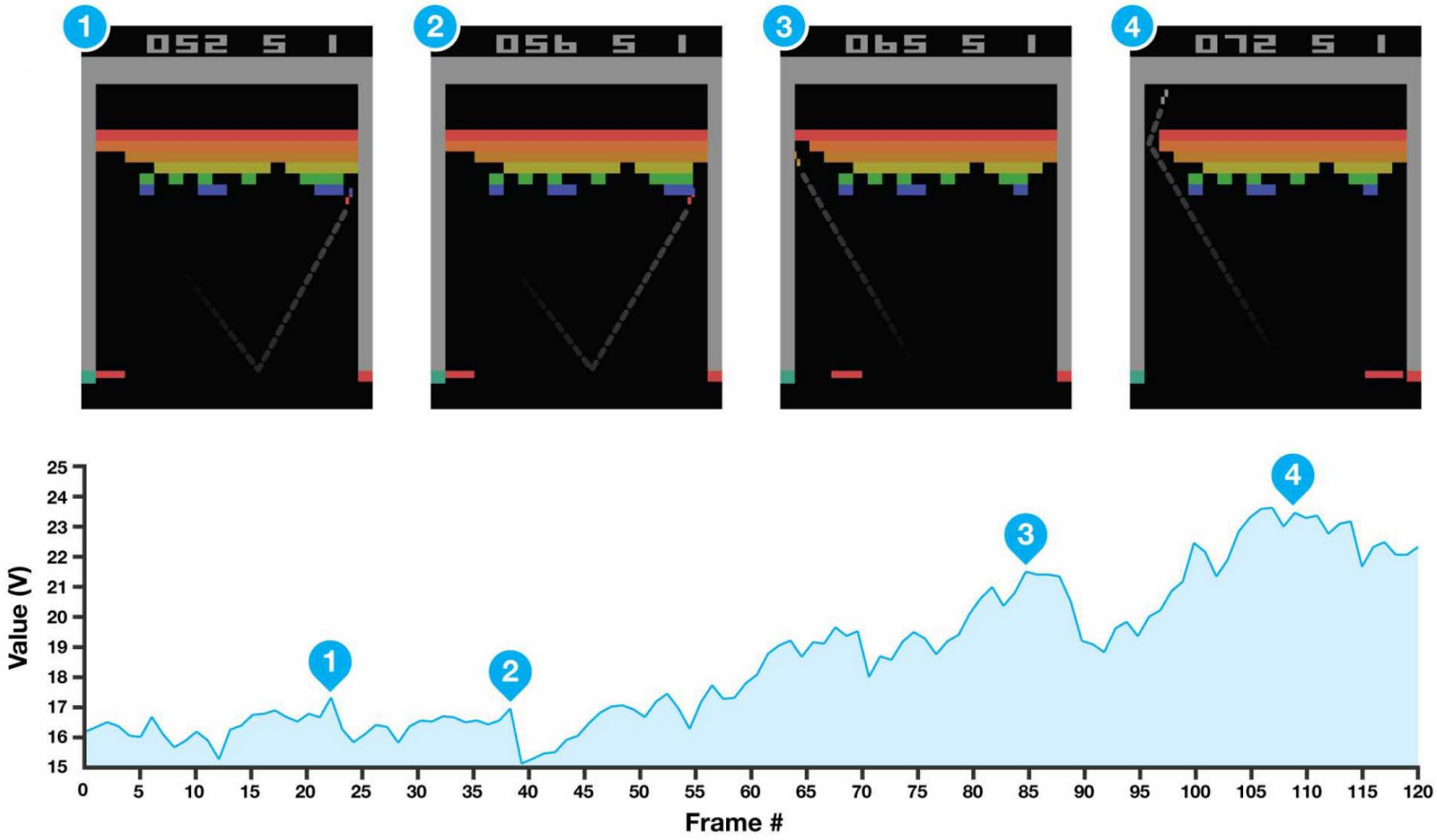
x_t

Imagem observada

y_j

Valores alvo

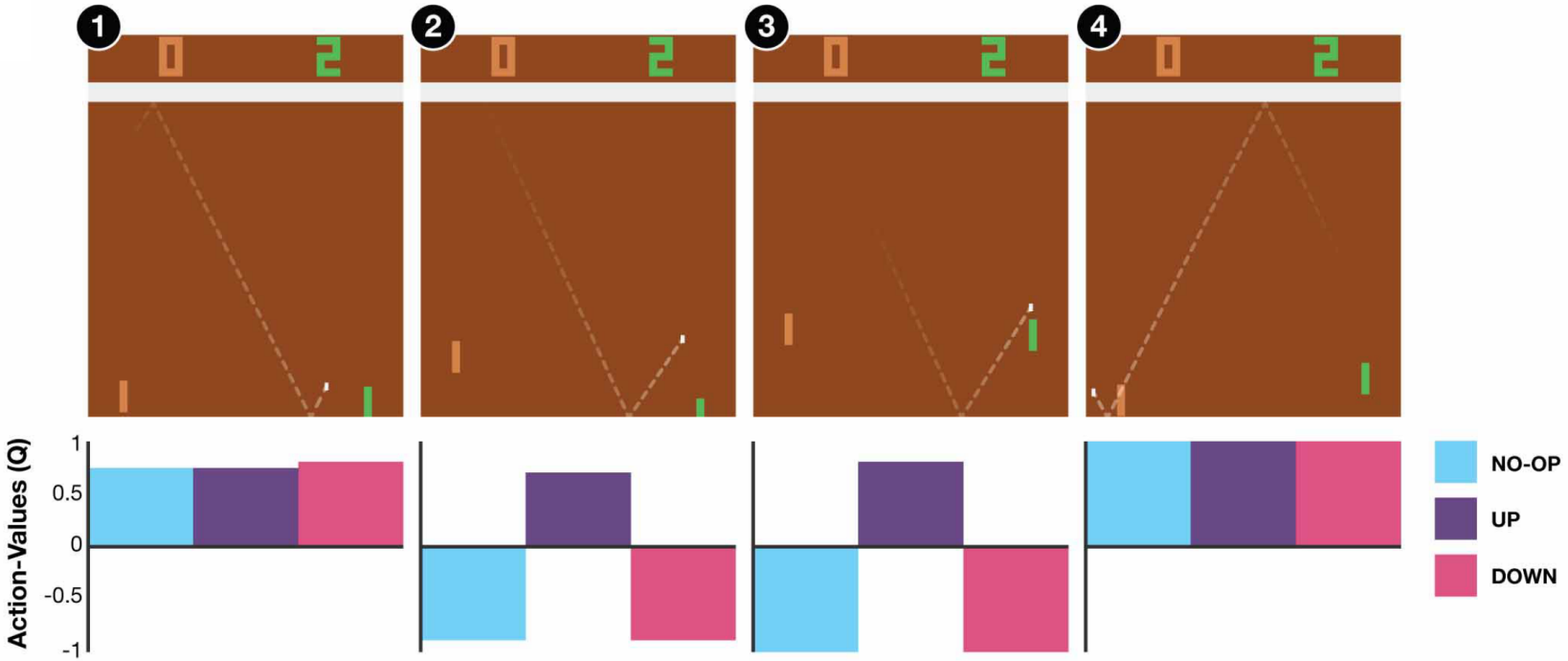
ALGORITMO *DEEP Q-LEARNING* – RESULTADOS OBTIDOS



[Mnih *et al.*, 2015]

Visualização da função valor $V(s)$ aprendida no jogo *Breakout* para vários situações do jogo

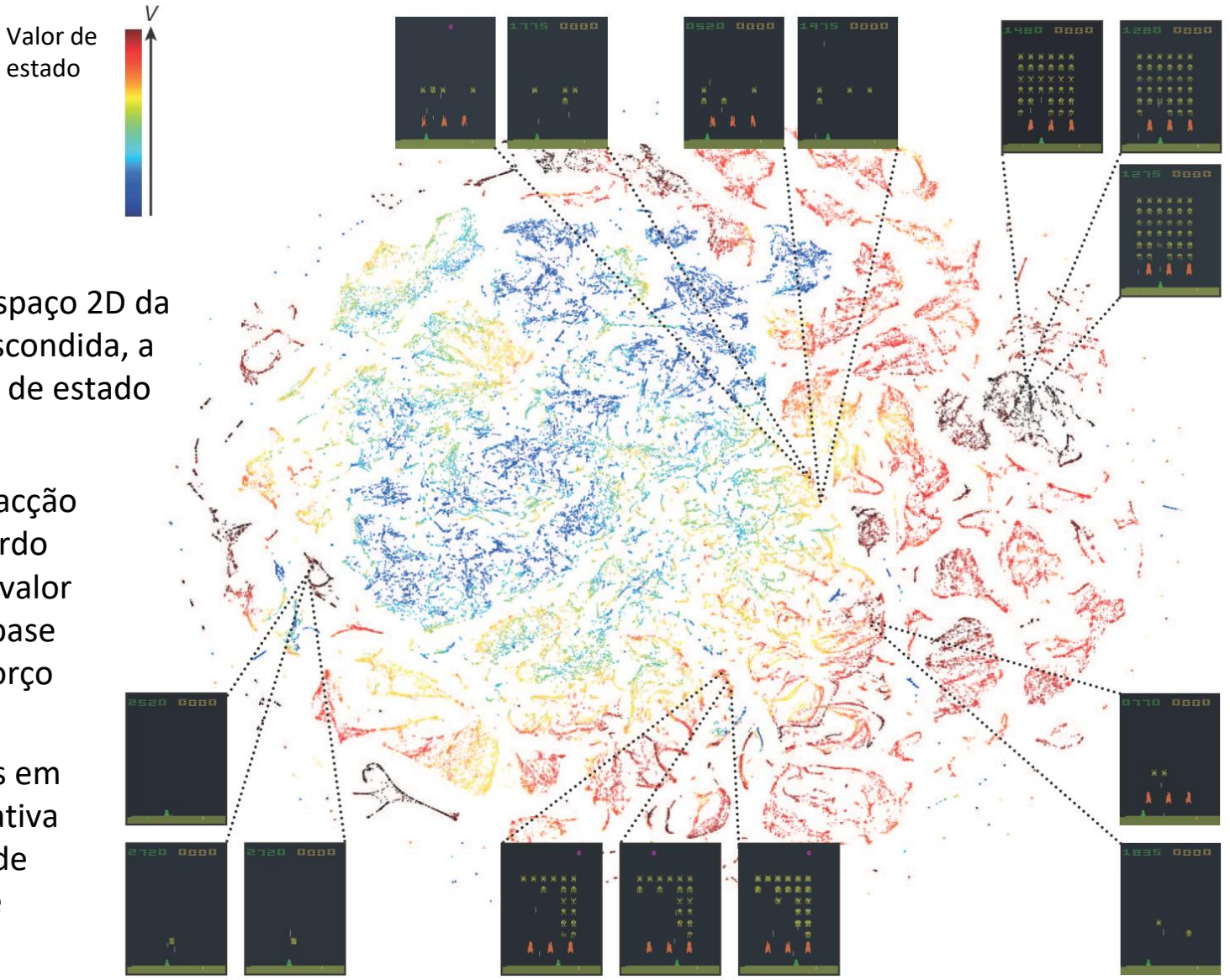
ALGORITMO *DEEP Q-LEARNING* – RESULTADOS OBTIDOS



[Mnih *et al*., 2015]

Visualização da função valor de estado-acção $Q(s,a)$ aprendida no jogo *Pong* para vários situações do jogo

ALGORITMO *DEEP Q-LEARNING* – RESULTADOS OBTIDOS



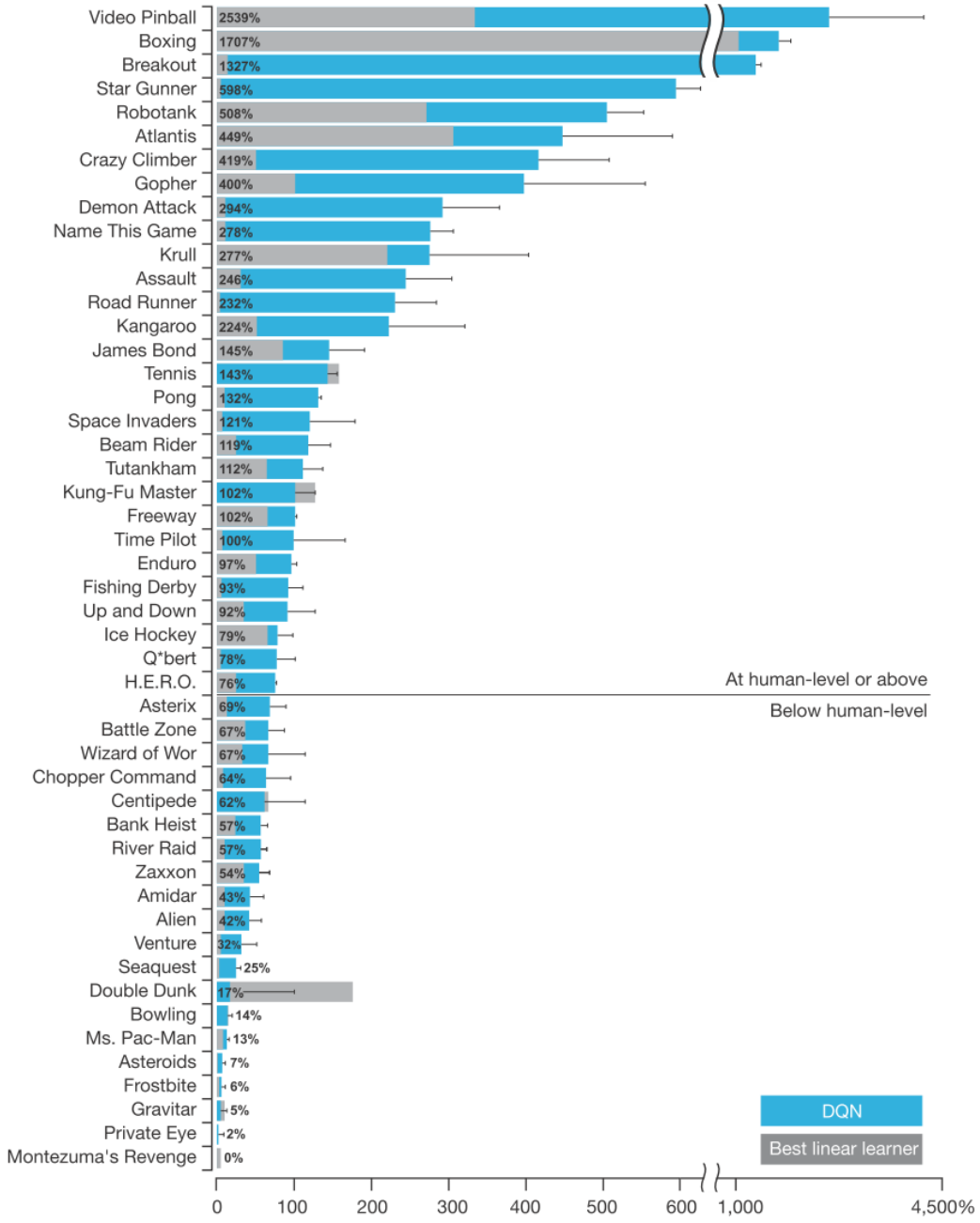
Projecção num espaço 2D da última camada escondida, a cor indica o valor de estado

A rede neuronal forma uma abstracção de estado de acordo com a função de valor aprendida (com base na função de reforço definida)

Estados próximos em termos de estimativa de valor, apesar de perceptualmente distintos

[Mnih *et al*., 2015]

ALGORITMO *DEEP Q-LEARNING* – RESULTADOS OBTIDOS



[Mnih *et al*., 2015]

REFERÊNCIAS

[Sutton & Barto, 2020]

R. Sutton, A. Barto, “Reinforcement Learning: An Introduction”, 2nd Edition, MIT Press, 2020

[Mnih *et al*., 2015]

V. Mnih, K. Kavukcuoglu, D. Silver *et al*. “Human-level control through deep reinforcement learning”, Nature 518, 529–533, 2015

[Kocabas, 2017]

M. Kocabas, “Human level control through deep reinforcement learning”, CENG793 Advanced Deep Learning Course, 2017

[Cai *et al.*, 2020]

S. Cai, S. Bileschi, E. Nielsen, F. Chollet, “Deep Learning with JavaScript: Neural networks in TensorFlow.js”, Manning Publications, 2020